

Phase 2 - Backend Development and Configurations

Project: Leave Management System (ServiceNow Scoped Application)

Scope: x_1958846_leave_0

1. Objective

Phase 2 is where the requirements from Phase 1 were translated into a self-enforcing system. The core design philosophy applied throughout this phase was: **the system, not the user, is responsible for correctness**. An employee should never be able to accidentally (or deliberately) submit a request with a wrong duration, a fabricated balance, or an unauthorized approval state because every one of those fields is either system-calculated, system-validated, or system-locked, layered across multiple points in the platform rather than relying on any single control.

This phase built the complete server-side and client-side automation layer: five Business Rules, three Client Scripts, two UI Policies, two UI Actions, and one Flow Designer flow each with a distinct, non-overlapping responsibility.

2. Data Model (as configured)

Leave Request

The transactional table employees interact with directly. Most of its fields are either auto-generated or read-only by design — of the fields visible on the form, only Requested By, Leave Type, Start Date, End Date, Half Day, and Comments are ever meant to be typed into by a human.

Field	Type	Notes
Number	String	Auto-generated (<code>LEAVEREQ000001</code>), read-only, sequential and zero-padded to avoid collision
Requested By	Reference → User	The employee submitting the request
Leave Type	Choice (CL, SL, EL, ML, WFH)	Casual, Sick, Earned, Maternity, Work From Home
Start Date / End Date	Date	Inclusive range
Half Day	True/False	When checked, forces Duration = 0.5 and End Date = Start Date

Field	Type	Notes
Duration	Floating Point Number	System-calculated, read-only — never manually entered
Status	Choice	Locked via UI Policy; reserved for future lifecycle states beyond approval
Approval	Choice (nyr, requested, approved, rejected)	Locked via UI Policy; changed only via UI Action or Flow Designer, never direct edit
Accured / Taken / Balance	Floating Point Number	Frozen audit snapshot copied from Leave Bucket at submission time, read-only
Comments	String	Conditionally mandatory when Duration exceeds 5 days

Leave Bucket

The live ledger — the single number both the employee and the system trust as "how much leave is actually left." Unlike the Leave Request snapshot, these values change continuously as requests are approved.

Field	Type	Notes
Employee	Reference → User	
Leave Type	Choice	Same choice list as Leave Request, kept in sync
Accured / Taken / Balance	Floating Point Number	The live, current ledger — updated automatically on approval
Last Year Balance	Floating Point Number	Supports carry-forward policy tracking across leave years

Leave Calculator

The policy configuration table HR uses to define entitlement — the layer that makes the system genuinely "global" rather than hardcoded to one country's leave rules.

Field	Type	Notes
Leave Type	Choice	
Starting Month / End Month	String	Defines the policy's validity period
Leaves	Floating Point Number	The entitlement quantity this policy grants
Country	Reference → Country	The field that makes leave policy country-specific
Leave Assignment	Choice	Controls how/when entitlement is applied to a bucket

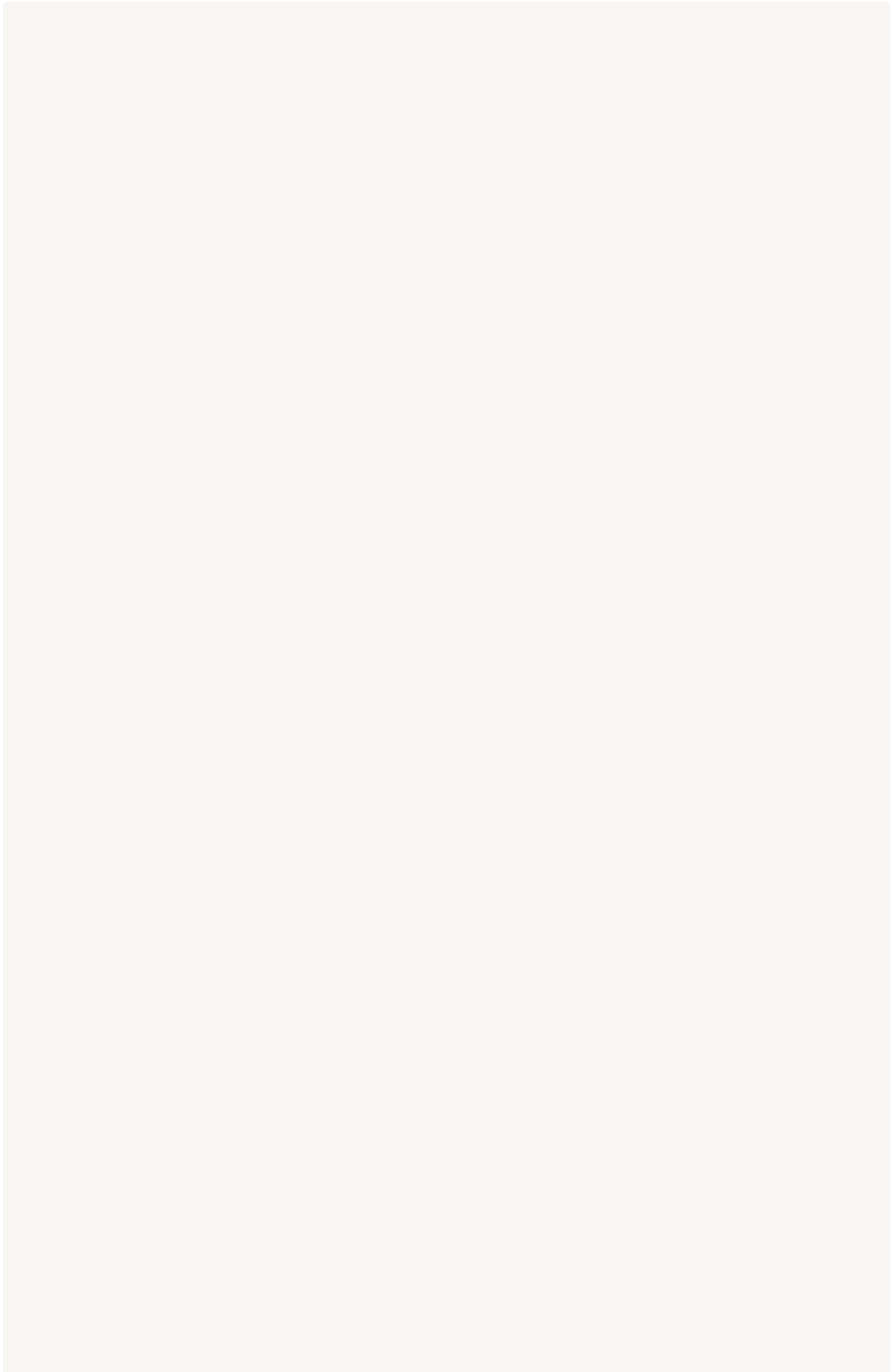
3. Business Rules

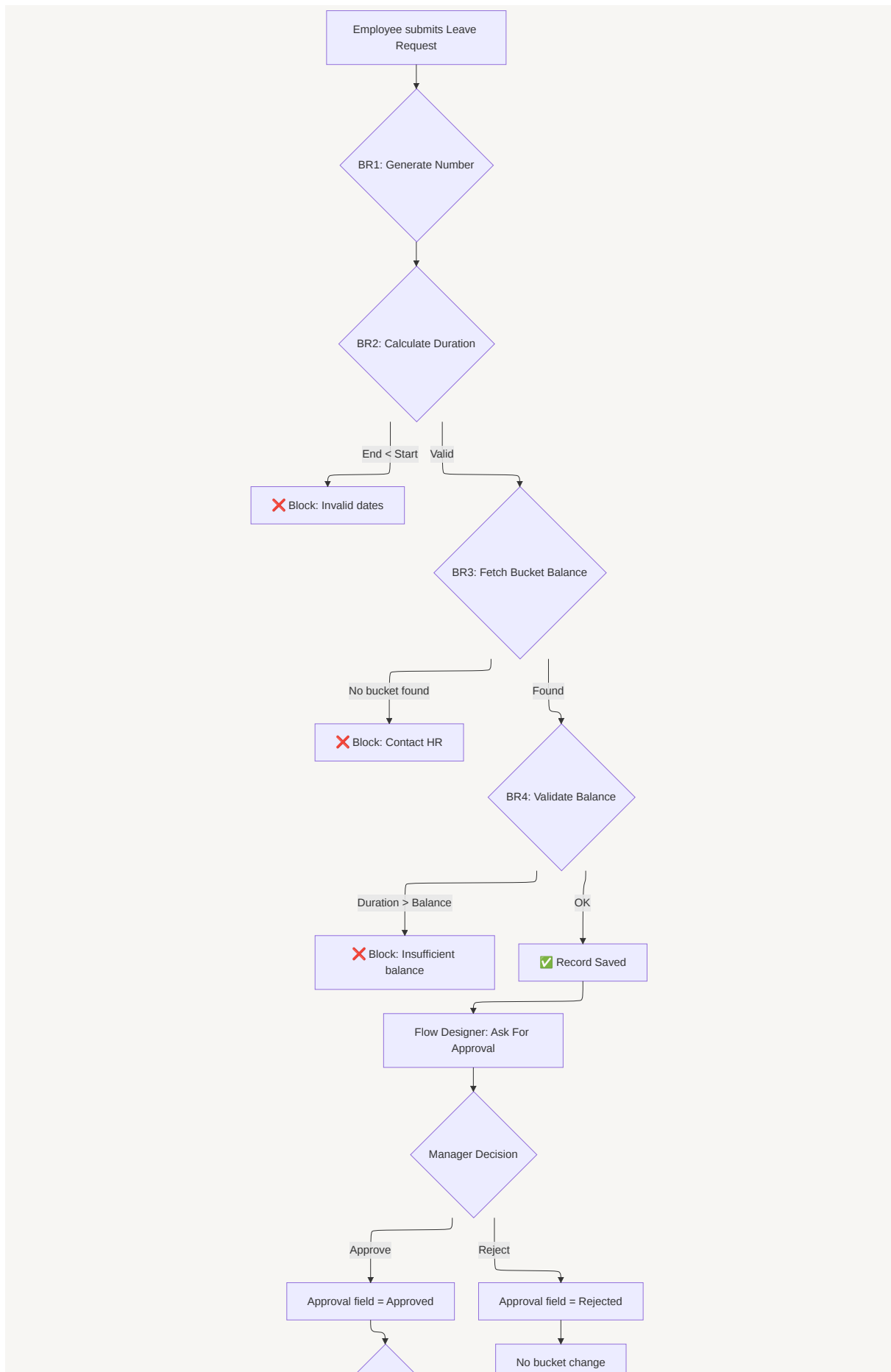
The five Business Rules form a deliberate, ordered pipeline. Order numbers were spaced in increments of 100 specifically so the execution sequence is self-documenting to anyone maintaining the app later — you can read the Order column top to bottom and understand the entire request lifecycle without opening a single script.

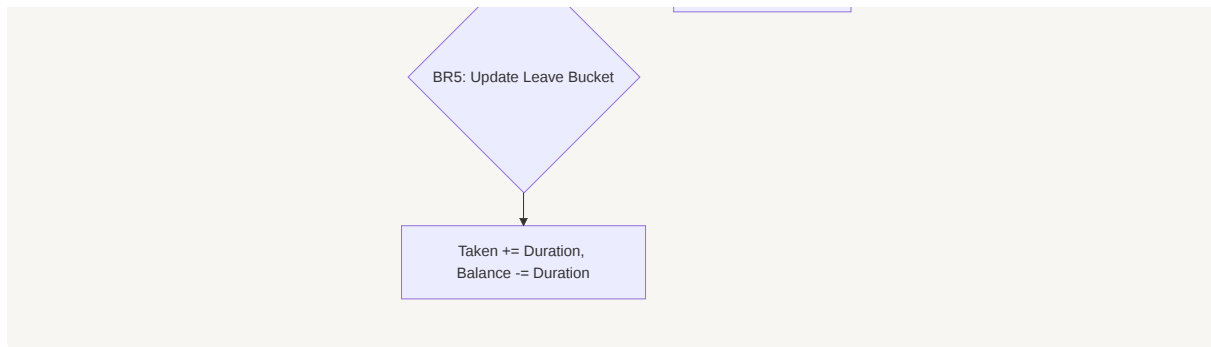
Order	Name	Table	When	Purpose
100	Auto Generate Leave Number	Leave Request	before insert	Generates a sequential <code>LEAVEREQ#####</code> number, safely padded to avoid collision even at high volume
200	Calculate Leave Duration	Leave Request	before insert/update	Computes Duration from Start/End Date and Half Day; blocks submission if End Date precedes Start Date
300	Fetch Balance from Leave Bucket	Leave Request	before insert/update (on relevant field change)	Copies Accrued/Taken/Balance from the matching Leave Bucket as a frozen snapshot; blocks submission entirely if no matching bucket exists for that employee/leave-type combination
400	Validate Sufficient Leave Balance	Leave Request	before insert/update	Blocks submission if the calculated Duration exceeds the snapshotted Balance
500	Update Leave Bucket on Approval	Leave Request	after update (Approval changes to Approved)	Deducts Duration from the Bucket's Taken/Balance — the only point in the entire system where the live ledger actually changes

Each rule was deliberately scoped to do exactly one job. This mattered in practice: because BR3 (balance fetch) is separate from BR4 (balance validation), either can be modified, tested, or debugged independently — for example, changing *how* a balance is fetched never risks breaking *how* it's validated, since they're different rules with different responsibilities.

Execution Flow







4. Client Scripts

Client Scripts in this application handle **only** user experience — none of them are relied upon for actual data integrity, since anything client-side can be bypassed (API calls, imports, background scripts). Every piece of real enforcement lives in the Business Rules above; the Client Scripts simply make the form pleasant and intuitive to use.

Name	Type	Field	Purpose
Toggle End Date on Half Day	onChange	Half Day	Hides the End Date field and visually mirrors Start Date the moment Half Day is checked, so the form never shows a misleading editable field
Preview Duration on Start Date Change	onChange	Start Date	Live client-side Duration preview, calculated instantly without a server round-trip
Preview Duration on End Date Change	onChange	End Date	Same live preview, triggered from the other end of the date range
Validate End Date Not Before Start Date	onChange	End Date	Gives an instant inline warning and auto-clears an invalid End Date, rather than letting the user discover the problem only after clicking Submit

A key technical lesson from this phase: all date math in Client Scripts uses the native JavaScript `Date` object, never `GlideDateTime` — `GlideDateTime` is a server-side-only class, and referencing it from an isolate/strict-mode Client Script throws a runtime error in the browser.

5. UI Policies

UI Policies were used wherever the requirement was purely declarative (show/hide, mandatory, read-only) with no calculation involved — reserving actual scripting for cases that genuinely needed it.

Name	Condition	Action
Lock Status and Approval fields	Always (no condition)	Status, Approval → Read-only, on every load, unconditionally
Require comments for long leave requests	Duration > 5	Comments → Mandatory, and automatically reverts to optional if Duration drops back to 5 or below

6. UI Actions

Name	Condition	Script Action
Approve	Admin/manager role AND Approval not already decided	Sets Approval = approved, saves the record, shows a confirmation message, and reloads
Reject	Admin/manager role AND Approval not already decided	Sets Approval = rejected, saves the record, shows a confirmation message, and reloads

These buttons serve as a controlled, auditable override path in addition to the primary Flow Designer-driven manager approval process described below — both paths converge on the same **Approval** field, which is what keeps Business Rule 500 blind to *how* an approval decision was made, and therefore reliable regardless of which path was used.

7. Flow Designer — Leave Request Approval Flow

This is the piece that makes the system feel genuinely automated rather than just validated. Without it, every request would need a human to manually notice it, open it, and click a button — Flow Designer removes that manual discovery step entirely.

Trigger: Leave Request record Created, where Approval is "Not yet Requested"

Action: Ask For Approval

- **Approvers:** dynamically dot-walked from **Trigger Record → Requested by → Manager** — meaning the flow was never hardcoded to a specific person; it resolves the correct approver at runtime for whichever employee submitted the request
- **Rule:** Anyone approves (the correct semantic choice for a single-approver scenario)
- **Approval Field:** bound directly to the custom **Approval** field, so the platform's native approval mechanism writes its outcome straight back into the application's own data model with no extra "Update Record" step required



8. Key Engineering Decisions & Rationale

Every non-obvious decision made during this phase is documented here, so future maintainers understand *why* something was built a certain way, not just *what* was built.

Decision	Rationale
Custom counter-based numbering instead of OOB auto-number	The table's Number field was originally created as a plain String, not a native auto-number field; solved cleanly with an order-100 Business Rule that pads and increments safely
Calendar-day duration (not working-day only)	A deliberate v1 business decision to keep the calculation simple and predictable; working-day/holiday exclusion is a documented future enhancement, not an oversight
Strict balance validation (block, don't auto-create bucket)	Chosen specifically to force clean HR data setup up front, preventing "phantom" balances from ever being silently invented by the system
Server-side (<code>GlideDateTime</code>) vs. client-side (<code>Date</code>) separation	A direct consequence of ServiceNow's isolate/strict-mode client script sandboxing, which blocks access to server-only Glide classes from the browser
Business Rules unaware of <i>how</i> Approval changed (UI Action vs. Flow)	A deliberate decoupled design — Business Rule 500 reacts purely to the resulting field state, which is what allows both the manual UI Action override and the automated Flow to coexist without duplicated or conflicting logic